PrivAR: Real-Time Privacy Protection for Location-Based Augmented Reality Applications

Shafizur Rahman Seeam
Rochester Institute of Technology
ss6365@rit.edu

Ye Zheng
Rochester Institute of Technology
yz7290@rit.edu

Zhengxiong Li
University of Colorado Denver
zhengxiong.li@ucdenver.edu

Yidan Hu
Rochester Institute of Technology
yidan.hu@rit.edu

***Abstract*—Location-based augmented reality (LB-AR) applications, such as Pokémon Go, stream sub-second GPS updates to deliver responsive and immersive user experiences. However, this high-frequency location reporting introduces serious privacy risks. Protecting privacy in LB-AR is significantly more challenging than in traditional location-based services (LBS), as it demands real-time location protection with strong per-location and trajectory-level privacy guaranteed while maintaining low latency and high quality of service (QoS). Existing methods fail to meet these combined demands.**

To fill the gap, we present PrivAR, the first client-side privacy framework for real-time LB-AR. PrivAR introduces two lightweight mechanisms: (i) Planar Staircase Mechanism (PSM) which designs a staircase-shaped distribution to generate noisy location with strong per-location privacy and low expected error; and (ii) Thresholded Reporting with PSM (TR-PSM), a selective scheme that releases a noisy location update only when a displacement exceeds a private threshold, enabling many-to-one mappings for enhanced trace-level privacy while preserving high QoS. We present theoretical analysis, extensive experiments on two public datasets and our proprietary GeoTrace dataset, and validate PrivAR on a Pokémon-Go-style prototype. Results show PrivAR improves QoS (Gamescore) by up to 50%, while increasing attacker error by 1.8x over baseline with an additional 0.06 milliseconds runtime overhead.

***Index Terms*—Location privacy, Location-based Augmented Reality, Geo-Indistinguishability**

I. Introduction

Augmented Reality (AR) overlays digital objects onto the physical world, transforming everyday spaces into immersive experiences. Location-Based AR (LB-AR) takes this further by integrating users' real-time location streams into the application (app) logic. Powered by platforms like ARKit [1], Google ARCore [2], and emerging wearable AR devices, like AR glasses [3], LB-AR has grown into a multi-billion-dollar industry, projected to surpass $19.7 billion by 2025 [4].

However, the real-time location data that fuels compelling LB-AR experiences also makes them uniquely invasive. Apps like Niantic's Pokémon Go stream locations up to 13 times per minute [5] to keep virtual content in lockstep with users. This high-frequency location update allows service providers (e.g., Niantic, Inc.) to store each user's feed as a detailed mobility log, sharply increasing the privacy risk. Public concern increased further after Niantic's controversial sale to a Saudi entity [6], illustrating the risks of mishandled LB-AR data, which created uncertainty for the data of nearly 200 million users. As backlash [7] grows and regulations tighten [8], robust real-time privacy protection for LB-AR systems has become imperative.

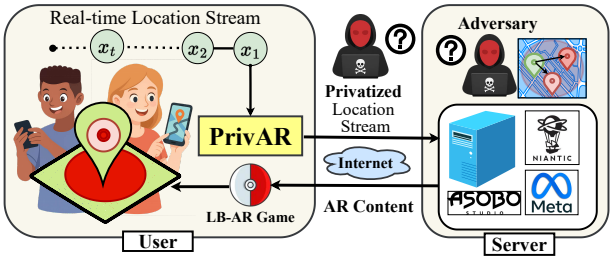


Fig. 1: End-to-end architecture of PrivAR-enabled LB-AR system.

Privacy protection in LB-AR presents far greater challenges than in traditional Location-Based Services (LBS). Traditional LBS, like place search (e.g., Yelp [9]) or advertising (e.g., GroundTruth [10]), issue low-frequency queries that can *tolerate higher latency and larger noise* without noticeably affecting the user experience. In contrast, LB-AR must stream sub-second location updates for immersive user experiences. This demands privacy mechanisms that meet stricter requirements: **R1**-Low latency: incurring sub-millisecond computational delay to support real-time location updates and responsive AR experiences, even on resource-constrained devices. **R2**- Rigorous privacy: providing real-time location protection with strong per-location and trace-level privacy guarantees, effectively defending against single-point and trajectory-level inference attacks on continuous streams; **R3**-High quality of service (QoS): keeping released privatized location with sufficiently low expected error to preserve AR functionality.

Unfortunately, mainstream location privacy mechanisms against untrusted service providers (e.g., LB-AR platforms like Niantic, Inc.) generally fail to simultaneously satisfy the three key requirements of LB-AR outlined above. Cryptographic protocols (e.g., [11]) achieve rigorous privacy but involve

heavyweight cryptographic primitives, violating **R1**. Spatial cloaking (e.g., [12]) is computationally lightweight yet offers only heuristic anonymity sets that are vulnerable to inference attacks, falls short of **R2**. Local Differential Privacy (LDP) mechanisms (e.g., [13]) are also lightweight and provide rigorous per-location and trace-level privacy, but introduce excessive noise to the released locations, failing to meet **R3**.

To address these limitations, Geo-indistinguishability (GeoInd) [14], a spatial adaptation of LDP, emerges as a compelling location privacy paradigm. By calibrating noise to geographic distance, GeoInd enables strong privacy with lower QoS loss, offering the potential to satisfy all three LB-AR requirements. Among existing GeoInd mechanisms [15]–[18], the Planar Laplace Mechanism (PLM) stands out for its simplicity and efficiency, making it a suitable choice for LB-AR scenarios that demand millisecond-scale processing latency. However, it falls short on **R2** and **R3** due to two critical limitations: First, PLM substantially distorts locations due to its biased polar implementation, incurring significant per-location QoS loss (See III-1). Second, applying PLM independently to perturb each location causes significant cumulative privacy degradation, failing to provide strong trace-level privacy guarantees (See III-2). These limitations underscore the need for GeoInd mechanisms that deliver millisecond-level latency, rigorous privacy guarantees, and high QoS—all at once—for real-time LB-AR.

We present PrivAR, the first client-side privacy framework for real-time LB-AR. PrivAR contributes two lightweight mechanisms: (i) Planar Staircase Mechanism (PSM), and (ii) Thresholded Reporting with PSM (TR-PSM). PSM samples noise from a well-designed staircase-shaped noise distribution that concentrates the probability mass closer to the true location, thereby reducing expected error and achieving higher per-location QoS than PLM. TR-PSM, inspired by the Sparse Vector Technique (SVT), addresses excessive trace-level privacy degradation by selectively perturbing location updates only when significant movement is detected, enabling many-to-one mappings that conserve the privacy budget and strengthen trajectory-level privacy. We summarize our contributions below:

- To the best of our knowledge, this is the first work to study privacy protection for the LB-AR systems.
- We propose PrivAR, a client-side privacy framework for the LB-AR system, which introduces two lightweight GeoInd mechanisms: (i) Planar Staircase Mechanism (PSM) providing rigorous location privacy with high QoS; and (ii) Thresholded Reporting with PSM (TR-PSM), which builds on PSM to enhance trace-level privacy protection with provable guarantee.
- Evaluation on two public datasets shows that PSM can achieve up to 2× QoS vs. PLM. Meanwhile, TR-PSM enhances privacy for trace (trajectory) data, increasing the Bayes risk of location inference attacks by up to 1.2×, effectively defending against sophisticated attacks.
- We developed an Android prototype game, inspired by Pokémon Go, to demonstrate the seamless integration of PrivAR into real-time LB-AR systems. Experiments on our collected dataset, *Geotrace*, show that PSM improves AR QoS (Gamescore) up to 50% of compared to baseline PLM. Meanwhile, TR-PSM significantly strengthens trace privacy, increasing the Bayes risk of inference attacks by up to 1.8×. Importantly, PrivAR adds a negligible 0.2% runtime overhead compared to a system with no privacy protection, demonstrating its practicality.

II. Preliminary

A. Geo-Indistinguishability

Definition 1 (ϵ-Geo-Indistinguishability (GeoInd) [14])**.** *A randomized mechanism $\mathcal{M}: \mathcal{X} \to \mathcal{Z}$ satisfies ϵ-GeoInd if for all input locations $x, x' \in \mathcal{X}$, and any possible output location $z \in \mathcal{Z}$:*

$$d_{\mathcal{P}}(\mathcal{M}(z,x), \mathcal{M}(z,x')) \leq \epsilon \cdot d(x,x') \tag{1}$$

where $d(x,x')$ is the Euclidean distance between x and x'.

B. Planar Laplace Mechanism

The Planar Laplace Mechanism (PLM) [14] achieves GeoInd on a continuous 2D plane by making a user's location $x \in \mathbb{R}^2$ indistinguishable within a radius r. It reports a noisy location $z \in \mathbb{R}^2$ drawn from probability density function (PDF) as:

$$\mathbf{D}_\epsilon(x)(z) = \frac{\epsilon^2}{2\pi} e^{-\epsilon d(x,z)} \tag{2}$$

where $\frac{\epsilon^2}{2\pi}$ normalizes the distribution and a larger ϵ implies weaker privacy.

C. Sparse Vector Technique

The Sparse Vector Technique (SVT) [19] answers a stream of threshold queries while incurring privacy cost only when the (noisy) result exceeds a noisy threshold. Assuming each query has unit sensitivity, SVT proceeds with two privacy parameters, ϵ_1 and ϵ_2, and a maximum of c positive answers:

$$\widetilde{T} = T + Laplace(1/\epsilon_1), \quad \widetilde{q_i} = q_i + Laplace(2c/\epsilon_2)$$

For each query q_i, SVT outputs *"above"* if $\widetilde{q_i} \geq \widetilde{T}$ and fewer than c positives have been returned so far; otherwise, it outputs *"below"*. SVT guarantees ϵ-DP with $\epsilon = \epsilon_1 + c \cdot \epsilon_2$.

D. Problem formulation

1) System Model: We consider a real-time LB-AR system where moderately imprecise location data does not compromise AR functionality, as in LB-AR games like *Pokémon GO* [20] and social platforms such as *Campfire* [21]. The system consists of three main components: a client device, a client application, and a remote server. The client, typically a smartphone or AR-capable wearable (e.g., AR glasses), is equipped with *trusted* [22] GPS sensors that reliably acquire the user's real-time location. The LB-AR client application (e.g., *Pokémon GO*) actively requests the user's real-time location at high frequency—typically every 4-5 seconds [5]—and continuously forwards the data to the remote server to fetch location-based AR content (e.g., specific objects in an LB-AR game) for real-time LB-AR service.

2) Adversarial Setting: We assume the client device is fully trusted and under the user's control. The LB-AR application, although developed by the service provider (i.e., the remote server), is assumed to execute only authorized operations as permitted by the user, without performing hidden data collection or unauthorized transmissions. It sends only user-approved location data to the server. In contrast, the remote server—which provides AR content and game logic—is modeled as an *honest-but-curious* adversary: it performs all functionalities correctly but is not trusted to preserve the privacy of user location data. It may accidentally leak location information or deliberately monetize it by sharing with third parties for commercial purposes [23]. Additionally, we consider a passive network-level eavesdropper capable of observing all outgoing real-time location data transmitted from the client to the server.

3) Design goals: We aim to design a *client-side Location Privacy Protection Mechanism (LPPM)* that perturbs each user's real-time location streams before transmission, with the following key design goals:

- **Rigorous Privacy.** The LPPM must provide strong privacy protection at both single-location and trajectory (trace) levels. Theoretically, it should satisfy ϵ-GeoInd privacy for each location release at a specific time slot, and ensure an overall ϵ'-GeoInd guarantee across the entire trajectory up to time T, thereby protecting against both point-level and trace-level inference. In practice, it should defend effectively against two representative attacks: i) *Single-location attack* — inferring a user's precise location from a single obfuscated report using Bayesian inference [24]; and ii) *Trace-based inference* — reconstructing user traces from an observed noisy location stream by exploiting spatio-temporal correlations across successive releases [25], [26].
- **High QoS.** The LPPM must preserve sufficient location accuracy to support meaningful AR interaction. While moderate localization noise is tolerable, it should maintain the expected error within a range that ensures high QoS.
- **Low Latency.** The LPPM must introduce minimal latency during per-location perturbation to ensure it meets the strict timing requirements of LB-AR applications without compromising responsiveness or user experience.
- **Seamless Deployability.** The LPPM must be lightweight and easily integrable into existing LB-AR apps, requiring minimal changes on the client side and *no modifications* to server-side logic or communication protocols.

III. Strawman approach and Limitations

A natural baseline for real-time location privacy in LB-AR is to apply PLM sequentially and independently to randomly perturb client's true location. Specifically, for each GPS location, x, we follow [14] to implement PLM efficiently by sampling noise in polar coordinates, where the radius r and angle θ follow:

$$\mathbf{D}_\epsilon(r,\theta) = \frac{\epsilon^2}{2\pi}\, r\, e^{-\epsilon r}, \qquad r \geq 0,\ \theta \in [0, 2\pi). \tag{3}$$

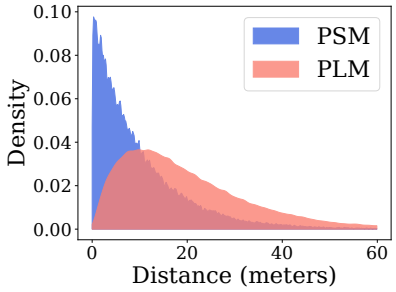


Fig. 2: Radial-noise profile when $\epsilon = 0.1$. PSM peaks probability near the origin, whereas PLM peaks $1/\epsilon$ meters away.

This PDF is the product of two independent random variables: the radius R and the angle Θ, where the PDF of R is defined by $D_{\epsilon,R}(r) = \epsilon^2 r e^{-\epsilon r}$, and the PDF of Θ $D_{\epsilon,\Theta}(\theta) = \frac{1}{2\pi}$. Next, we randomly generate r and θ according to the corresponding PDFs. The reported (perturbed) location, z, is then given by $z = x + (r\cos\theta, r\sin\theta)$.

PLM satisfies ϵ-GeoInd [14] and can generate perturbed locations in a few milliseconds (see Table. III), making it a practical choice for real-time, on-device deployment in LB-AR systems. However, independently applying PLM to each location introduces two key limitations.

*1) **L1:** Per-location QoS Loss:* To enable efficient sampling, PLM transforms the planar Laplace distribution (Eq. 2) into its polar form (Eq. 3), introducing a Jacobian term r. This term shifts the peak probability density away from the true location, concentrating it around $r = 1/\epsilon$. As a result, perturbed points several meters away from the true location become more likely than those near it. For example, as shown in Fig. 2, under $\epsilon = 0.1$, the PLM distribution (in salmon pink) peaks near 10 meters from the origin, indicating that points farther from the true location are most likely to be reported. This bias increases expected error [17], degrading the QoS of individual location reports.

*2) **L2:** Excessive Trace-Level Privacy Degradation:* Applying PLM independently to each location in a trajectory leads to significant privacy degradation in high-frequency location-based services such as LB-AR, both in theory and in practice.

a) Cumulative Budget Exhaustion: PLM incurs a separate privacy loss at each timestamp, leading to cumulative budget exhaustion over time. Formally, for a trajectory $\mathbf{x} = \{x_1, \ldots, x_T\}$ up to time slot T, applying PLM independently to each location x_t yields a composite mechanism $\texttt{IM}(\mathbf{x}) = [\mathcal{M}_\epsilon(x_1), \ldots, \mathcal{M}_\epsilon(x_T)]$, where each $\mathcal{M}_\epsilon$ satisfies ϵ-GeoInd [14]. According to the basic composition property of GeoInd, the cumulative privacy loss of IM increases linearly with the number of reported locations, resulting in an overall privacy guarantee of $T \cdot \epsilon$-GeoInd [14]. This linear budget exhaustion either forces the per-location privacy budget ϵ to be reduced as T increases, leading to more noise and reduced QoS, or leads to significantly weakened trajectory-level privacy.

b) Increased Risk of Trace Inference: PLM adds independent noise at each timestamp, resulting in a one-to-

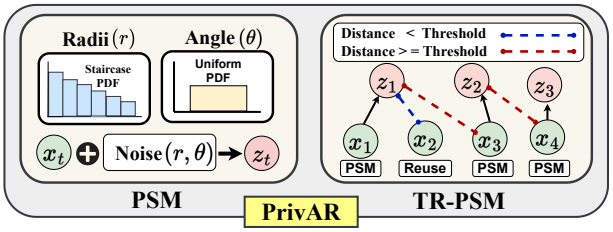


Fig. 3: PrivAR overview: PSM adds noise per-location; TR-PSM reuses the last noisy location unless displacement exceeds privatized threshold.

one mapping $x_t \to z_t$ and statistically independent reports. This increases vulnerability to trace inference for two main reasons. First, in LB-AR settings, the high sampling rate means adjacent true locations are often very close. Independently perturbing each point causes the noisy outputs to cluster tightly around the true trajectory, enabling adversaries to accurately infer the path. Recent studies report that under such repeated perturbations, 80–93% of users' top-1 locations can be reidentified [26]. Second, the lack of temporal ambiguity inherent in independently reported locations further amplifies their vulnerability to trace inference attacks.

These limitations render PLM unsuitable for LB-AR, highlighting the need for a client-side LPPM that ensures robust privacy, high QoS, low latency, and easy deployment.

IV. Privacy for LB-AR Applications

This section presents PrivAR, a client-side privacy framework that addresses the baseline's limitations. We first introduce the Planar Staircase Mechanism (PSM) to address *L1: per-location QoS degradation*. Next, building on PSM, we propose Thresholded Reporting with PSM (TR-PSM) to further mitigate *L2: trace-level degradation*.

A. Planar Staircase Mechanism

We first propose PSM to enhance per-location QoS while providing the same level of privacy protection as PLM by addressing the peak shifting issue in PLM. Specifically, PSM adopts a staircase-shaped probability distribution that concentrates more probability density near the origin, thereby reducing expected error and improving utility. It preserves the same formal GeoInd guarantee and lightweight design as PLM, executing within milliseconds on representative devices (see Table III), while significantly improving QoS. We now describe the construction of PSM.

1) Mechanism Design: Let $x \in \mathbb{R}^2$ be the true location. PSM generates the perturbed location $z \in \mathbb{R}^2$ as:

$$z = x + r(\cos\theta, \sin\theta),$$

where $r \geq 0$ is sampled from a staircase-shaped distribution $f_r(r)$ parameterized by ϵ, and the angle θ is sampled from uniformly from $\theta \sim \text{Unif}(0, 2\pi)$, independently of r.

The PDF $f_r(r)$ is defined by partitioning the radial domain into intervals of width $\Delta \in \mathbb{R}^+$. Formally, for each interval indexed by $i \in \mathbb{N}^+$:

Algorithm 1: Planar Staircase Mechanism

Input: True point $x \in \mathbb{R}^2$, privacy budget ϵ
Output: Perturbed point $z \in \mathbb{R}^2$

$u \sim \text{Unif}(0, 1)$ // sample uniform noise
$r \leftarrow C_\epsilon^{-1}(u)$ // sample staircase radius
$\theta \sim \text{Unif}(0, 2\pi)$ // sample isotropic angle
$z \leftarrow x + r(\cos\theta,\ \sin\theta)$ // add planar noise
return z

$$f_r(r) = \begin{cases} f_1(r), & 0 \leq r \leq \Delta \\ f_2(r), & \Delta < r \leq 2\Delta \\ \vdots & \vdots \\ f_n(r), & (n-1)\Delta < r \leq n\Delta \end{cases}$$

Each interval $f_i(r)$ is constant, forming a staircase:

$$f_i(r) = \frac{(1 - e^{-\epsilon})e^{-(i-1)\epsilon}}{(1 - e^{-n\epsilon})\Delta} \tag{4}$$

Typically, we set $\Delta = 1$ for finer granularity; n is the number of intervals, set to be large ($n \to \infty$) to approximate an unbounded distribution similar to PLM. Thus: $f_i(r) = (1 - e^{-\epsilon})e^{-(i-1)\epsilon}/\Delta, \quad i = 1, 2, \ldots$

2) Sampling Procedure: We sample the radial distance r using the inverse CDF method, following [14]. The radial CDF $C_\epsilon(r)$ is given by:

$$C_\epsilon(r) = \sum_{i=1}^{k-1} \int_{(i-1)\Delta}^{i\Delta} f_i(t)\, dt \ + \int_{(k-1)\Delta}^{r} f_k(t)\, dt, \tag{5}$$

where k denotes the interval such that $(k-1)\Delta < r \leq k\Delta$.

Given $t \sim \text{Unif}(0, 1)$, we solve $r = C_\epsilon^{-1}(t)$. This inverse can be computed either (i) via numerical interpolation (e.g., using *numpy.interp* on a precomputed CDF table), or (ii) analytically via the closed-form inverse. The full procedure is detailed in Algorithm 1.

As shown in Fig. 2 (blue area), PSM concentrates probability density closer to the true location. Empirical results in Table I further confirms that PSM consistently achieves lower mean and maximum displacement, demonstrating superior QoS under the same privacy guarantee.

Theorem 1. PSM *satisfies ϵ-GeoInd for each location.*

Proof. Let $x, x' \in \mathcal{X}$ be any two possible locations in the Cartesian plane. Denote by $\mathcal{M}$ the PSM mechanism with a large number of radial intervals n. We show that $\mathcal{M}$ satisfies ϵ-GeoInd, i.e., for any infinitesimally small region $S \subseteq \mathcal{Z}$ around some output point z, $\frac{\Pr[\mathcal{M}(x) \in S]}{\Pr[\mathcal{M}(x') \in S]} \leq e^{\epsilon d(x, x')}$.

To find the worst-case ratio, note that the probability of $\mathcal{M}(x)$ falling in S is proportional to the PDF value at $\|z - x\|$, and similarly for $\mathcal{M}(x')$. Since the radial PDF $f_r(r)$ is a decreasing function of r, the ratio is maximized when S is centered at $z = x$, where $\mathcal{M}(x)$ achieves its maximum and

TABLE I: Empirical displacement error under PLM and PSM.

Mechanism	Mean Distance (m)	Max Distance (m)
PLM	19.99	173.34
PSM	**10.05**	**159.46**

$\mathcal{M}(x')$ its minimum. Also notice that for any $d(x, x')$, there exists an integer k such that $(k-1)\Delta < d(x, x') \leq k\Delta$. The worst-case privacy loss is then:

$$\frac{\Pr[\mathcal{M}(x) \in S]}{\Pr[\mathcal{M}(x') \in S]} = \frac{p_{\mathcal{M}(x)}(x)}{p_{\mathcal{M}(x')}(x)} = \frac{f_1(r)}{f_k(r)}$$
$$= \frac{(1-e^{-\epsilon})e^0}{(1-e^{-\epsilon})e^{-(k-1)\epsilon}} = e^{(k-1)\epsilon} \leq e^{\epsilon d(x,x')}.$$

Therefore, the theorem is proved. □

Trace-Level Privacy. While PSM ensures rigorous location privacy at each timestamp, applying it independently to every update causes the privacy loss to accumulate linearly over time, leaving *L2* unresolved (see Section III-2) and rendering PSM ineffective for high-frequency LB-AR applications that stream location updates at short intervals.

B. Thresholded Reporting with Planar Staircase Mechanism

*1) **Overview**:* To address *L2:excessive trace-level privacy degradation*, we propose Thresholded Reporting with PSM (TR-PSM), a selective perturbation mechanism inspired by Sparse Vector Technique (SVT) [19]. SVT was originally introduced for threshold-based query monitoring in differential privacy. SVT allows a system to selectively respond only to "significant" queries — those that exceed a noisy threshold — while suppressing or reusing responses when changes are small. This approach greatly reduces the number of times the privacy budget is consumed, even across long query sequences.

We borrow this idea to streaming location updates by treating each location as a query. Specifically, we monitor the distance between the current true location and the last released (perturbed) one, and release a new perturbed location only if that distance exceeds a privatized threshold $\tilde{\delta}$. Otherwise, we simply reuse the last released location z_{ref}, avoiding unnecessary privacy budget consumption. This approach effectively: 1) Conserves the privacy budget by spending it only when significant location changes occur, thereby mitigating cumulative budget privacy loss; 2) Reduces unnecessary perturbation of near-stationary positions, avoiding overlapping noise clouds that can leak trajectory information through output clustering, thus strengthening protection against trace inference attacks.

*2) **Detailed Design**:* Before a session begins, the client specifies the desired privacy level ϵ_T for the entire session. Suppose the client at x_t for each time slot t, TR-PSM then performs the following steps to ensure rigorous privacy protection for each individual location and the overall trajectory throughout the session.

When $t = 1$, TR-PSM draws random noise $\eta \sim \mathcal{M}_r(\epsilon)$ from PSM's radial sampler and sets $\tilde{\delta} = \delta + \eta$, where δ is an app-chosen parameter (e.g., 5 m). This randomization prevents adversaries from accurately predicting future release points. $\tilde{\delta}$ is stored and reused throughout the session. TR-PSM also perturbs the first location x_1 using PSM, $z_1 = \mathcal{M}(x_1, \epsilon)$, publishes z_1, and sets $z_{\text{ref}} \leftarrow z_1$ to track the most recent reported location. z_{ref} is maintained and updated throughout the session.

When $t \geq 2$, for each new location x_t, the client computes the distance between x_t and z_{ref}, $d_t = \|x_t - z_{\text{ref}}\|$. Based on this distance, we conditionally decide whether to release a new perturbed point (incurring privacy cost) or reuse the previous one (at no cost). Specifically, as shown in Fig. 3, if $d_t \geq \tilde{\delta}$, PSM is applied to randomly perturb x_t, yielding $z_t \leftarrow \mathcal{M}(x_t, \epsilon)$, and the reference point is updated as $z_{\text{ref}} \leftarrow z_t$; otherwise, the previous location is reused, i.e., $z_t \leftarrow z_{\text{ref}}$.

TR-PSM will keep tracking the total privacy budget during the session. Let k be the number of threshold crossings up to time slot t. If the total cost $(k+2)\epsilon \geq \epsilon_T$, TR-PSM will provide a *BudgetExhausted* warning to the client. The client either enlarges the total privacy budget to continue the session or just terminate it and use the LB-AR later in a new session. The whole procedure is encapsulated in Algorithm 2.

Assume that the user stops the session after time T, and the true location stream up to time T is $\mathbf{x} = \{x_1, \cdots, x_T\}$. TR-PSM can provide the following privacy protection.

Theorem 2. *TR-PSM satisfies ϵ-GeoInd for each location and ϵ_T-GeoInd for a trace $\mathbf{x}$, where $\epsilon_T = (k+2)\epsilon$ and k is the number of threshold crossings after the first fix.*

Proof. At each time step t, if the distance between the current location x_t and the last released location z_{ref} exceeds the privatized threshold $\tilde{\delta}$, the mechanism releases a new noisy location using PSM. Since PSM satisfies ϵ-GeoInd [14], each such release incurs a privacy loss of ϵ. Otherwise, TR-PSM reuses z_{ref}, which does not incur additional privacy loss, satisfying 0-GeoInd.

Over the course of the session, a total of $k+1$ locations are perturbed using PSM—one at initialization ($t = 1$) and k more due to threshold crossings. By the sequential composition property of GeoInd [14], these contribute $(k+1)\epsilon$ to the total privacy loss. Additionally, sampling the privatized threshold $\tilde{\delta}$ at session start cost an additional ϵ. Therefore, the total privacy loss for the trace $\mathbf{x}$ is $\epsilon_T = (k+2)\epsilon$, terminating the proof. □

Discussion. We discuss three aspects of TR-PSM.

Choosing the Displacement Threshold. The threshold δ must exceed typical GPS jitter to prevent wasting privacy budget on minor, noise-induced movements. A smaller δ causes frequent, unnecessary releases, while a larger δ can overly suppress updates, degrading QoS. Table I further illustrates how δ influences QoS across datasets, highlighting that optimal values vary depending on mobility patterns and update frequency. Hence, δ is empirically tuned based on the app's location update rate and users' movement characteristics.

Budget Conservation. Each session incurs a fixed *setup* cost of 2ϵ (one for privatized threshold; one for the first release), plus ϵ for every subsequent releases. If k threshold crossings occur after the first release, the total privacy cost is: $\epsilon_T = 2\epsilon + k\epsilon =$

Algorithm 2: Threshold Reporting with PSM

Input: Trace $\mathbf{x} = (x_1, \ldots, x_T)$; budgets ϵ_T, ϵ; threshold δ
Output: Stream $\mathbf{z} = (z_1, \ldots, z_T)$

/* Init */
if $\epsilon_T < 2\epsilon$ **then**
  return error `// budget too small`
$\eta \sim \mathcal{M}_r(\epsilon)$; $\tilde{\delta} \leftarrow \delta + \eta$ `// noisy threshold`
$z_1 \leftarrow \mathcal{M}(x_1, \epsilon)$; $z_{\text{ref}} \leftarrow z_1$
$\epsilon_{\text{left}} \leftarrow \epsilon_T - 2\epsilon$
for $t \leftarrow 2$ **to** T **do**
  $d_t \leftarrow \|x_t - z_{\text{ref}}\|$
  if $d_t < \tilde{\delta}$ **then**
    $z_t \leftarrow z_{\text{ref}}$ `// reuse`
  else
    if $\epsilon_{left} < \epsilon$ **then**
      return BudgetExhausted
    $z_t \leftarrow \mathcal{M}(x_t, \epsilon)$; $z_{\text{ref}} \leftarrow z_t$; $\epsilon_{\text{left}} \leftarrow \epsilon_{\text{left}} - \epsilon$
return $\mathbf{z}$

$(k+2)\epsilon$. In contrast, applying independent perturbation at every timestamp (as in PLM) would cost $T \cdot \epsilon$. Thus, TR-PSM saves $(T-k-2)\epsilon$ whenever $k+2 < T$, which is typical in high-frequency location streams.

Runtime Overhead. Each new location triggers only: (i) a simple constant-time distance check and (ii) at most one additional PSM sampling operation if the threshold is crossed. Thus, TR-PSM remains lightweight, on par with PLM/PSM.

V. Simulation Studies

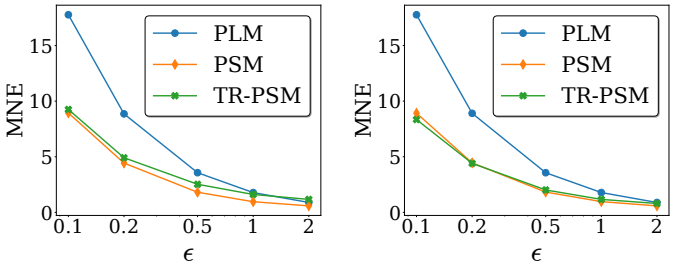
We first evaluate the proposed solutions in terms of QoS, privacy, and latency using public datasets.

A. Experimental Setting

1) Environment: QoS and privacy evaluations were run on an M2 MacBook Pro (16GB RAM), while latency was measured across three Android phones (low-end, mid-range, and high-end). All results are averaged over 100 trials.

2) Datasets: We evaluate our methods on two real-world datasets: Geolife [27] and T-drive [28]. We use sub-sampled versions of Geolife and T-drive. Geolife contains 1.05 million points covering 23,061 km, with a median step of 8 meters and 2-second intervals. T-drive includes 809K points across 4,343 km, with a 49-meter median step and 181-second intervals.

3) Preprocessing: Following prior work [24], [29], we define a 6×6 km^2 area and discretize it into a 200×200 grid. Each true location $x_t = (\phi_t, \lambda_t)$, where ϕ_t and λ_t denote latitude and longitude, is mapped to a corresponding grid cell $s_t \in \mathcal{S}$. The LPPM then perturbs x_t to produce a noisy location z_t. The resulting (z_t, s_t) pairs are used to train the attack model.



(a) Geolife (b) T-drive

Fig. 4: QoS comparison on Geolife and T-drive: Mean Normalized Error (MNE) vs. ϵ (0.1–2) for PLM, PSM, TR-PSM; lower is better.

4) Attack model: We consider the *trace-based inference attack* [29], in which the adversary observes a sequence of perturbed locations $\{z_1, z_2, \ldots, z_T\}$ and exploits spatio-temporal correlations to infer the user's true trace. Following [24], [29], we adopt the k-Nearest Neighbors (k-NN) as the inference model, setting $k = \log(T)$. This choice is motivated by the universal consistency of k-NN, which guarantees convergence as sample size $T \to \infty$ [30].

5) Evaluation Metrics: Our evaluation includes:

a) Quality of Service (QoS).: We quantify distortion using Mean Normalized Error (MNE) [31], which measures the average distance between true and perturbed locations across all traces. Let $\mathcal{T}$ denote the set of traces, with each trace $\tau_j = \{(x_t^{(j)}, z_t^{(j)})\}_{t=1}^{T_j}$. Then:

$$\text{MNE} = \frac{1}{|\mathcal{T}|} \sum_{j=1}^{|\mathcal{T}|} \frac{1}{T_j} \sum_{t=1}^{T_j} d(x_t^{(j)}, z_t^{(j)}) \tag{6}$$

where $d(\cdot, \cdot)$ denotes the Euclidean distance. Lower MNE implies better QoS by indicating smaller spatial error [17].

b) Privacy: We estimate the adversary's success using empirical Bayes risk [32]. Given training samples $\{(z_t, s_t)\}_{t=1}^{T}$, where z_t is the perturbed location and s_t the true grid cell, we train an inference function f_T to predict s from z. The estimated Bayes risk is:

$$\hat{\mathcal{R}}_{f_T} := 1 - \sum_{z \in \mathcal{Z}} \max_{s \in \mathcal{S}} \Pr(z, s)$$

where $\Pr(z, s) = \Pr(z \mid s)\pi(s)$ denotes the joint distribution. Higher $\hat{\mathcal{R}}_{f_T}$ indicates stronger privacy, as it reflects greater uncertainty in the adversary's inference.

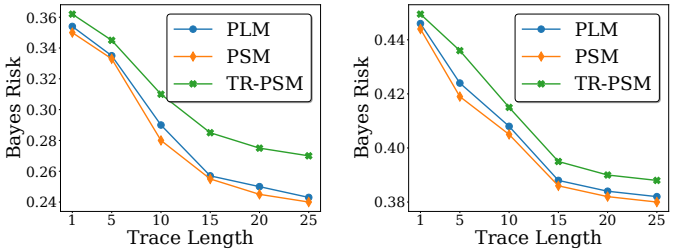
c) Latency: We measure latency as the average time required to perturb each location. Lower latency implies the mechanism is viable for real-time LB-AR applications.

B. Simulation results

1) QoS: Fig. 4 reports the MNE of PLM, PSM, and TR-PSM across two datasets, with ϵ ranging from 0.1 to 2. As expected, MNE decreases with increasing ϵ for all mechanisms, since higher privacy budgets introduce less noise, producing perturbed locations closer to the ground truth. PSM consistently achieves the lowest MNE, demonstrating superior QoS—especially under strong privacy settings (i.e., low ϵ).

TABLE II: Mean Normalized Error (MNE) of TR-PSM for $\epsilon \in \{0.1, 1\}$ over varying δ.

	$\epsilon = 0.1$					$\epsilon = 1$				
Dataset	$\delta = 5$	$\delta = 10$	$\delta = 20$	$\delta = 50$	$\delta = 100$	$\delta = 5$	$\delta = 10$	$\delta = 20$	$\delta = 50$	$\delta = 100$
Geolife	9.48	10.61	14.23	26.49	48.01	1.74	3.33	7.41	20.26	42.20
Tdrive	8.44	8.65	9.51	11.48	14.84	1.17	1.54	2.31	4.20	7.54



(a) Geolife (b) T-drive

Fig. 5: Trace-level privacy Geolife and T-drive: Bayes risk of inference attack vs. trajectory length at $\epsilon = 0.1$; higher is better.

TABLE III: Per-fix perturbation runtime (ms) on mobile devices.

Device	**PLM**	**PSM**	**TR-PSM**
Galaxy A04 (Low-end)	0.556	0.559	0.964
Pixel 6a (Mid-range)	0.114	0.112	0.152
Galaxy S22 (High-end)	0.054	0.055	0.069

In contrast, PLM yields the highest MNE, particularly at lower privacy budgets. While TR-PSM incurs slightly higher MNE than PSM, it still significantly outperforms PLM. More importantly, this modest QoS loss is offset by the substantial privacy gains TR-PSM provides (see V-B2).

2) Privacy: Figure 5 reports the estimated Bayes risk of PLM, PSM, and TR-PSM at a fixed privacy budget $\epsilon = 0.1$ as trace length increases from 1 to 25 across two datasets. Here, trace length 1 represents single-location attack. As expected, Bayes risk declines with longer traces, reflecting greater adversarial inference power due to accumulating spatio-temporal correlations. However, TR-PSM consistently mitigates this degradation through its many-to-one strategy, in which multiple true locations are mapped to the same reported point, reducing distinguishability and confusing the adversary. For example, in Fig. 5a, the Geolife dataset—with a median inter-fix distance of approximately 8 meters—exhibits rapid privacy loss under PLM and PSM due to dense, highly correlated movement patterns. In contrast, TR-PSM maintains higher Bayes risk by reusing prior outputs, thereby breaking sequential correlations. In the sparser T-drive dataset (median inter-fix distance = 49 meters; Fig. 5b), where spatial continuity is weaker, all mechanisms degrade more gradually and the adversary's inference advantage is inherently limited.

3) Latency: Table III reports the mean runtime per location update across three Android devices. PLM and PSM exhibit nearly identical runtimes, as both apply single-shot perturbation with comparable computational cost. TR-PSM introduces slightly higher overhead due to its conditional logic, but still achieves millisecond-level latency on all tested devices. Even on the low-end Galaxy A04, TR-PSM completes perturbation within 0.1ms, while on modern phones like the Pixel 6a and Galaxy S22, it operates well below 0.02ms. These latencies are orders of magnitude lower than the 16.7ms display refresh interval of standard 60 Hz screens—a common threshold influencing perceptible responsiveness in AR rendering [33]. All mechanisms thus comfortably satisfy real-time performance requirements for latency-sensitive AR applications.

VI. PrivAR: Implementation and Evaluation

This section details the end-to-end implementation of PrivAR within a custom-built Pokémon GO–inspired app, demonstrating its operation in a full LB-AR system. We then evaluate its performance in a real-world use case.

A. End-to-End Implementation of LB-AR app

This section outlines PrivAR's integration and end-to-end client–server workflow in a custom-built LB-AR app.

1) Client-Side Workflow: The client-side app was developed in Kotlin 2.0.21 using Android Studio Meerkat (2023.3.1) with Android Gradle Plugin 8.9.0. It targets Android 7.0 (API level 24) or higher and is compiled against SDK 35 with Java 11 compatibility. The app leverages the Fused Location Provider API (Play Services Maps v19.1.0) to capture precise locations, which are perturbed in real time by PrivAR before server transmission. PrivAR is implemented as a self-contained Kotlin plugin and is designed for open-source release as a lightweight, ARCore-compatible library. Its core mechanisms, PSM and TR-PSM, were initially prototyped in Python (NumPy 1.21.5, SciPy 1.7.3, Pandas 1.4.2) and reimplemented in Kotlin for efficient on-device deployment.

App Interface. The app provides an intuitive user interface (UI) with separate configuration and gameplay modes. In the configuration UI, users choose a privacy mechanism (PLM, PSM, or TR-PSM) and set a privacy level: $\epsilon = 0.5$ (low), $\epsilon = 0.2$ (medium), or $\epsilon = 0.1$ (high). While three preset levels are provided for simplicity, PrivAR supports any $\epsilon \geq 0$. The baseline PLM is included alongside PrivAR's PSM and TR-PSM for comparison. During gameplay, the app displays an interactive map with two concentric 100-meter visibility regions centered on the true and perturbed locations. Their intersection defines the region where virtual objects are rendered. A dynamic counter indicates the proportion of game objects falling within this shared region.

2) Server-Side Workflow: The backend server is a lightweight AR content generator implemented in Flask v3.1. Hosted on a local machine (MacBook Pro), it exposes a RESTful API and communicates with the mobile client

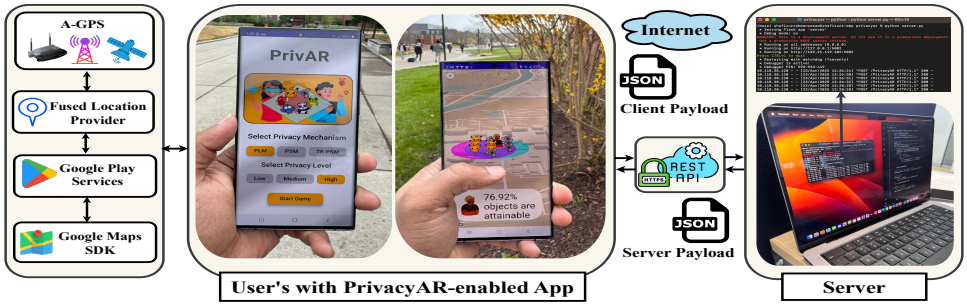


Fig. 6: In our prototype PrivAR-enabled app, each device perturbs the user's real-time GPS coordinates according to the selected mechanism before sending them to the game server. The server ingests these noisy locations, computes the positions of virtual objects in response (e.g., nearby PokéStops), and returns their coordinates in a compact JSON payload. Upon receipt, the client parses that payload and renders the virtual content—delivering a seamless, low-latency AR experience.

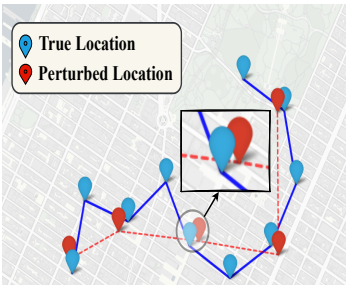


Fig. 7: TR-PSM releases a new noisy location (red) only when movement from the last report exceeds a privatized threshold; otherwise, the previous location is reused.

(Samsung Galaxy S22) over Wi-Fi. The server listens for HTTP requests at the */PrivAR* endpoint, parses incoming JSON payloads, extracts the perturbed location, and dynamically generates nearby virtual game objects.

3) Client–Server Workflow: Gameplay begins when the user selects privacy preferences—e.g., **PSM** with **High**—via the configuration UI. Upon tapping **Start**, the app continuously acquires real-time locations using the Fused Location Provider API. Each location is perturbed using the selected mechanism, and both the true and perturbed coordinates[1] are packaged into a JSON payload and sent to the server via HTTP Post. The server processes the perturbed location, generates virtual game objects nearby, and returns them in a structured JSON response. The client then renders only the objects located within the overlapping visibility regions.

Figure 6 shows an end-to-end implementation of PrivAR, and Figure 7 shows TR-PSM in action.

Integration in Commercial Apps. PrivAR is designed for seamless drop-in integration as a standalone client-side library, requiring no modifications to server logic or communication protocols. It is fully compatible with existing LB-AR infrastructures, enabling commercial apps to enhance location privacy with minimal engineering overhead.

B. Practicality of PrivAR

We evaluate the practicality of PrivAR through real-world experiments, focusing on AR-specific QoS, robustness against location inference, and end-to-end latency in a client–server workflow with PrivAR integrated.

1) Experimental Setup: Five participants (3 male, 2 female) were each provided with a preconfigured Samsung Galaxy S22 Ultra running our prototype app with PrivAR integrated. Participants completed multiple sessions of varying durations, freely moving within designated areas while observing app responsiveness. During each session, the app received JSON payloads from the server to render virtual objects based on perturbed locations, and internally logged key QoS metrics

[1]In real-world deployment, only the perturbed location is transmitted; the true location is shown here solely to evaluate AR quality under privacy

TABLE IV: Latency breakdown of client–server workflow.

Operation	Duration (ms)
Device acquires location fix	0.03
Apply PrivAR (PSM/TR-PSM)	**0.05–0.07**
Serialize JSON payload	1.32
Server-side object generation	0.28
Network round-trip (request + response)	30.52
Parse JSON response and render object	1.70
Total End-to-End Latency	33.90–33.92

along with ground-truth mobility traces for post-analysis. In total, we collected approximately 127 km of trajectory data across diverse mobility modes—walking, running, biking, and driving—forming our *GeoTrace* dataset, which will be released upon paper acceptance. On the server side, virtual objects were uniformly generated around perturbed locations under two spatial densities: sparse (100 m) and dense (50 m).

2) Evaluation Metrics: In typical AR games, players capture virtual objects via throwing mechanics (e.g., Poké Balls in Pokémon GO) or proximity-based interactions. In our design, we simplify this by assuming users automatically capture all virtual objects within their visibility region. We evaluate QoS in AR (Gamescore) using two metrics:

(i) *Catchable Objects*: the percentage of virtual objects still reachable after perturbation:

$$\mathcal{C}_{\text{obj}} = \frac{|\mathcal{V} \cap \hat{\mathcal{V}}|}{|\mathcal{V}|} \times 100\%$$

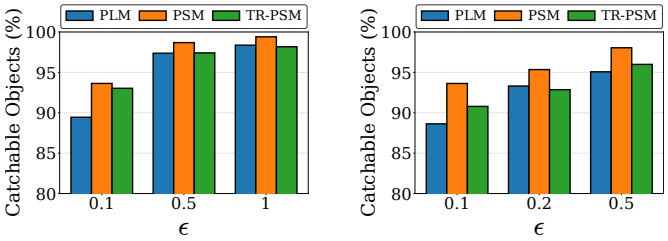
where $\mathcal{V}$ and $\hat{\mathcal{V}}$ denote object sets around the true and perturbed locations.

(ii) *Accumulated Loss*: the total number of objects lost during the session:

$$\mathcal{L}_{\text{acc}} = \sum_{t=1}^{T} \left(|\mathcal{V}^{(t)}| - |\mathcal{V}^{(t)} \cap \hat{\mathcal{V}}^{(t)}| \right)$$

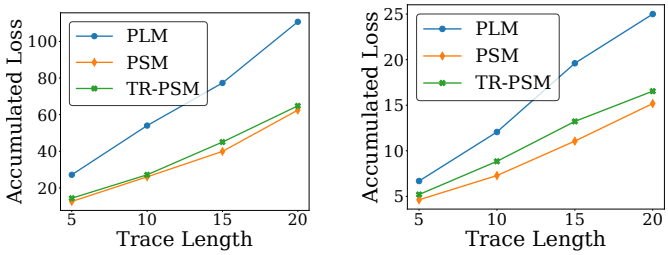
where T is the session length and $\mathcal{V}^{(t)}$ denotes the object set at time t.

3) Evaluation: We describe the results below.



(a) Dense Virtual Objects (b) Sparse Virtual Objects

Fig. 8: % Catchable objects vs. ϵ (0.1-0.5) under (a) dense (b) and sparse virtual-object layouts; higher is better.



(a) Dense Virtual Objects (b) Sparse Virtual Objects

Fig. 9: Accumulated loss vs. trace length when $\epsilon = 0.1$ under (a) dense (b) and sparse virtual-object layouts; lower is better.

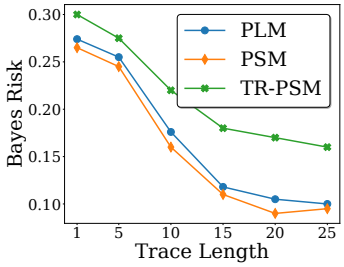


Fig. 10: Bayes risk of trace inference attack for *GeoTrace* at $\epsilon = 0.1$.

a) QoS in AR Gameplay : As shown in Fig. 8 and Fig. 9, both PSM and TR-PSM outperform PLM across evaluated privacy levels. Notably, TR-PSM preserves high object catchability (see Fig. 8) even under strong privacy constraints (e.g., $\epsilon = 0.1$) compared to PLM, while PSM consistently outperforms others. Similarly, in terms of accumulated loss over gameplay sessions, TR-PSM reduces object loss by 30–50% and PSM by 35–57%, depending on object density.

b) Privacy: As shown in Fig. 10, TR-PSM significantly enhances privacy relative to both PLM and PSM, particularly in highly correlated traces such as our collected *GeoTrace*. By reusing previously released noisy locations, it disrupts one-to-one mappings between true and reported positions, hindering trajectory reconstruction and improving Bayes risk by 1.8×.

c) Latency: We evaluate end-to-end latency across the full client–server pipeline. As shown in Table IV, our privacy mechanisms (PSM/TR-PSM) contributes only 0.05–0.07 ms per location perturbation. This represents less than 0.2% of the total end-to-end latency. In contrast, the bulk of the delay arises from non-privacy components such as network round-trips (30.52 ms), JSON handling (3 ms). While the total latency slightly exceeds the 16 ms limit [33] for AR responsiveness, it is dominated by system-level overheads unrelated to perturbation. Crucially, our privacy mechanisms are not the bottleneck and is fully compatible with real-time AR constraints.

C. Summary of Properties

- **Robust Privacy.** Both mechanisms satisfy formal privacy guarantees; TR-PSM improves Bayes risk by up to 1.8×.
- **High QoS.** PSM halves MNE versus PLM; TR-PSM achieves strong privacy with minor accuracy trade-off.
- **Low Latency.** End-to-end latency remains within real-time thresholds, with $< 0.2\%$ added cost.
- **Seamless Integration.** PrivAR requires no changes to AR logic or protocols, enabling drop-in deployment.

VII. Related Work

Location-Privacy Mechanisms. Cryptographic protocols [11]) provide strong guarantees but incur latency and heavy mobile CPU use, unsuitable for resource-constrained mobile devices. Spatial cloaking hides a user in a k-anonymity region [12], but lacks formal guarantees and is vulnerable to composition attacks [34]. Obfuscation with Bayesian adversary tuning [35] maximises privacy for a given prior, yet degrades sharply when the prior changes. LDP eliminates the trusted server but injects large noise [13], harming user-centric QoS in mobile AR [36]. Geo-Indistinguishability (GeoInd) [14] offers an elegant solution for location protection; its Planar Laplace Mechanism (PLM) perturbs each fix in $\mathcal{O}(1)$ time and is therefore the de-facto baseline for mobile scenarios. Optimal GeoInd solutions [15]–[17] reduce error but require solving large linear programs or enumerating the output space, prohibitive for on-device LB-AR. Our PSM inherits PLM's closed-form sampling efficiency while mitigating radial bias, and TR-PSM further addresses GeoInd's trace fragility.

Trajectory-Level Privacy. Offline DP publication of whole traces [37]–[39] yields strong guarantees but presupposes the entire trajectory—a non-starter for live AR. Online adaptations exploit Markov models within a *predefined* map [40]–[42]; unpredictable open-world motion in LB-AR breaks their assumptions. Recent correlation attacks recover up to 93% of traces [26] even after single-fix location privacy.

Privacy in Mobile/AR Systems. AR research has focused on access control prompts [43] and TEEs/isolated execution [44]–[47]. These protect code and memory but not the live GPS stream itself, leaving apps to implement their own LPPM. To our knowledge, no prior work offers a practical, client-side location-privacy layer tailored to high-frequency LB-AR.

VIII. Conclusion

We presented PrivAR, the first fully client-side privacy framework tailored to the real-time demands of LB-AR systems. PrivAR introduces two lightweight yet effective mechanisms. First, PSM introduces a novel staircase-shaped

noise distribution that concentrates probability mass near the true location, reducing expected error for enhanced per-location QoS while maintaining GeoInd guarantees equivalent to PLM. Second, TR-PSM selectively releases location updates only when displacement exceeds a privatized threshold, effectively reducing redundant transmissions and enhancing trace-level privacy through many-to-one mappings. Through extensive evaluation on two public datasets and our GeoTrace dataset, we demonstrate that PSM improves QoS in AR (Gamescore) by up to 50% over PLM, while TR-PSM increases attacker Bayes risk by up to 1.8×, offering stronger resistance to inference attacks. Importantly, both mechanisms introduce only 0.05–0.07 milliseconds of runtime overhead per location update—less than 0.2% of the end-to-end latency—ensuring seamless integration in real-time mobile environments.

REFERENCES

[1] "Arkit 6- augmented reality," https://developer.apple.com/augmented-reality/arkit/.

[2] "Arcore - google for developers," https://developers.google.com/ar.

[3] "Hololens," https://www.microsoft.com/en-us/hololens.

[4] I. Forum, "Location-based augmented reality: Top things to know in 2024," https://theinfluencerforum.com/location-based-augmented-reality-in-2024/.

[5] "The creators of pokémon go mapped the world. now they're mapping you," https://kotaku.com/the-creators-of-pokemon-go-mapped-the-world-now-theyre-1838974714, 2019.

[6] J. Koebler, "Saudi arabia buys pokémon go, and probably all of your location data," 2024, https://www.404media.co/saudi-arabia-buys-pokemon-go-and-probably-all-of-your-location-data/.

[7] "The dark side of Pokemon Go ," Available Online:https://www.cbsnews.com/news/the-dark-side-of-pokemon-go/, [Accessed 31-July-2025].

[8] "General Data Protection Regulation GDPR," Available Online:https://www.nist.gov/privacy-framework, [Accessed 7-Nov-2022].

[9] Yelp, "Yelp: Restaurants, dentists, bars, beauty salons, doctors," https://www.yelp.com/, 2025.

[10] Groundtruth, "The advertising platform that drives in-store visits and other real business results." https://www.groundtruth.com/, 2025.

[11] J. Shao, R. Lu, and X. Lin, "Fine: A fine-grained privacy-preserving location-based service framework for mobile devices," in *IEEE INFOCOM 2014 - IEEE Conference on Computer Communications*, 2014, pp. 244–252.

[12] N. Li, T. Li, and S. Venkatasubramanian, "t-closeness: Privacy beyond k-anonymity and l-diversity," in *IEEE 23rd International Conference on Data Engineering*, 2007, pp. 106–115.

[13] H. Wang, H. Hong, L. Xiong, Z. Qin, and Y. Hong, "L-srr: Local differential privacy for location-based services with staircase randomized response," in *Proceedings of the 2022 ACM SIGSAC Conference on computer and communications security*, 2022, pp. 2809–2823.

[14] M. E. Andrés, N. E. Bordenabe, K. Chatzikokolakis, and C. Palamidessi, "Geo-indistinguishability: Differential privacy for location-based systems," in *CCS '13*, 2013, p. 901–914.

[15] N. E. Bordenabe, K. Chatzikokolakis, and C. Palamidessi, "Optimal geo-indistinguishable mechanisms for location privacy," ser. CCS '14, 2014, p. 251–262.

[16] U. I. Atmaca, S. Biswas, C. Maple, and C. Palamidessi, "A privacy-preserving querying mechanism with high utility for electric vehicles," *IEEE Open Journal of Vehicular Technology*, vol. 5, pp. 262–277, 2024.

[17] S. Biswas and C. Palamidessi, "Privic: A privacy-preserving method for incremental collection of location data," *arXiv preprint arXiv:2206.10525*, 2022.

[18] K. Chatzikokolakis, C. Palamidessi, and M. Stronati, "Constructing elastic distinguishability metrics for location privacy," *arXiv preprint arXiv:1503.00756*, 2015.

[19] M. Lyu, D. Su, and N. Li, "Understanding the sparse vector technique for differential privacy," *arXiv preprint arXiv:1603.01699*, 2016.

[20] I. Niantic, "Pokémon go," https://pokemongolive.com/en/.

[21] "Campfire," https://campfire.nianticlabs.com/en/, 2022.

[22] N. O. Tippenhauer, C. Pöpper, K. B. Rasmussen, and S. Capkun, "On the requirements for successful gps spoofing attacks," in *Proceedings of the 18th ACM Conference on Computer and Communications Security*, ser. CCS '11, 2011, p. 75–86.

[23] "Ftc takes action against gravy analytics, venntel for unlawfully selling location data tracking consumers to sensitive sites," https://tinyurl.com/4jw9but6, 2024.

[24] G. Cherubin, K. Chatzikokolakis, and C. Palamidessi, "F-bleau: fast black-box leakage estimation," in *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2019, pp. 835–852.

[25] Y. Dong, Y. Hu, A. Aseeri, D. Li, and R. Zhang, "Location inference under temporal correlation," in *2023 32nd International Conference on Computer Communications and Networks (ICCCN)*. IEEE, 2023, pp. 1–10.

[26] L. Yu, S. Zhang, Y. Meng, S. Du, Y. Chen, Y. Ren, and H. Zhu, "Privacy-preserving location-based advertising via longitudinal geo-indistinguishability," *IEEE Transactions on Mobile Computing*, vol. 23, no. 8, pp. 8256–8273, 2023.

[27] Y. Zheng, X. Xie, and W.-Y. Ma, "Understanding mobility based on gps data," in *Proceedings of the 10th ACM conference on Ubiquitous Computing (Ubicomp 2008)*, September 2008.

[28] Y. Zheng, "T-drive trajectory data sample," August 2011.

[29] S. R. Seeam, Z. Li, and Y. Hu, "A black-box approach for quantifying leakage of trace-based correlated data," in *The 7th International Workshop on Physics Embedded AI Solutions in Mobile Computing (MobiCom Picasso Workshop)*, 2024.

[30] C. J. Stone, "Consistent nonparametric regression," *The annals of statistics*, pp. 595–620, 1977.

[31] Y. Zhang, Q. Ye, R. Chen, H. Hu, and Q. Han, "Trajectory data collection with local differential privacy," *Proc. VLDB Endow.*, vol. 16, no. 10, p. 2591–2604, Jun 2023.

[32] M. Romanelli, K. Chatzikokolakis, C. Palamidessi, and P. Piantanida, "Estimating g-leakage via machine learning," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 697–716.

[33] J. Deber, R. Jota, C. Forlines, and D. Wigdor, "How much faster is fast enough? user perception of latency & latency improvements in direct and indirect touch," in *Proceedings of the 33rd annual acm conference on human factors in computing systems*, 2015, pp. 1827–1836.

[34] S. R. Ganta, S. P. Kasiviswanathan, and A. Smith, "Composition attacks and auxiliary information in data privacy," in *Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining*, 2008, pp. 265–273.

[35] R. Shokri, G. Theodorakopoulos, C. Troncoso, J.-P. Hubaux, and J.-Y. Le Boudec, "Protecting location privacy: optimal strategy against localization attacks," in *the ACM conference on Computer and communications security*, 2012, pp. 617–627.

[36] A. Athanasiou, K. Chatzikokolakis, and C. Palamidessi, "Enhancing metric privacy with a shuffler," in *PETS 2025-25th Privacy Enhancing Technologies Symposium*, 2025.

[37] F. Tian, S. Zhang, L. Lu, H. Liu, and X. Gui, "A novel personalized differential privacy mechanism for trajectory data publication," in *2017 International Conference on Networking and Network Applications (NaNA)*. IEEE, 2017, pp. 61–68.

[38] T. Cunningham, G. Cormode, H. Ferhatosmanoglu, and D. Srivastava, "Real-world trajectory sharing with local differential privacy," *arXiv preprint arXiv:2108.02084*, 2021.

[39] Y. Zhang, Q. Ye, R. Chen, H. Hu, and Q. Han, "Trajectory data collection with local differential privacy," *arXiv preprint arXiv:2307.09339*, 2023.

[40] Y. Xiao and L. Xiong, "Protecting locations with differential privacy under temporal correlations," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 2015, pp. 1298–1309.

[41] W. Zhang, M. Li, R. Tandon, and H. Li, "Online location trace privacy: An information theoretic approach," *IEEE Transactions on Information Forensics and Security*, vol. 14, no. 1, pp. 235–250, 2019.

[42] Y. Cao, M. Yoshikawa, Y. Xiao, and L. Xiong, "Quantifying differential privacy under temporal correlations," in *2017 IEEE 33rd International Conference on Data Engineering (ICDE)*. IEEE, 2017, pp. 821–832.

[43] S. Jana, D. Molnar, A. Moshchuk, A. Dunn, B. Livshits, H. J. Wang, and E. Ofek, "Enabling fine-grained permissions for augmented reality applications with recognizers," in *USENIX Security*, 2013, pp. 415–430.

[44] V. Costan and S. Devadas, "Intel sgx explained," *Cryptology ePrint Archive*, 2016.

[45] Y. Shen, H. Tian, Y. Chen, K. Chen, R. Wang, Y. Xu, Y. Xia, and S. Yan, "Occlum: Secure and efficient multitasking inside a single enclave of intel sgx," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 955–970.

[46] J. R. Sanchez Vicarte, B. Schreiber, R. Paccagnella, and C. W. Fletcher, "Game of threads: Enabling asynchronous poisoning attacks," in *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*, 2020, pp. 35–52.

[47] P. Ratazzi, A. Bommisetti, N. Ji, and W. Du, "Pinpoint: Efficient and effective resource isolation for mobile security and privacy," *arXiv preprint arXiv:1901.07732*, 2019.